

Automation: Work Queues (Foundations)

Power CAT

Module 05: Cloud flow advanced — work queues (core lab)



This is the required core lab: Track A, modules A1–A8, plus the concepts, setup, and appendices you need. The optional self-exploration modules B1–B8 are in the companion file `05-b-work-queues.md`.

How this lab is structured

This lab is delivered in two tracks so it fits both a timed session and self-paced exploration.

The end-to-end work queue — capture once, prioritize, dequeue in the cloud, execute, and invoice

Figure: The end-to-end work queue — capture once, prioritize, dequeue in the cloud, execute, and invoice.

Track	Modules	Status	What you get
A — Mandatory	A1–A8	Required	A complete, runnable cloud work queue: create, enqueue in priority order, dequeue, approve, invoice, and mark Processed.
B — Self-exploration	B1–B8	Optional	Every remaining work queue capability: schema validation, timeouts, exception handling, retries, human remediation, SLA, concurrency, and ALM depth.



Setup check: Complete **Track A in order** — each module builds on the last. **Track B modules are independent;** do them in any order that respects the *Builds on* note at the top of each. Every Track B module extends the flows from Track A, so finish Track A first.

Prerequisites

The scenario continues the earlier modules, but the build stands alone — you can complete this lab without them. You need:

- A Power Platform development environment where you have maker permissions. Dataverse requires premium capacity; a free [developer environment](#) covers everything in this lab.
- A **Power Automate Premium license or trial**(see Appendix L). Work queues are a premium capability.
- The **Environment Maker** security role, or another role that includes access to the work queue tables (see Appendix K).
- The **Northwind Traders** solution imported into that environment **and seeded with sample data** (see [solutions/README.md](#)).
- An email-capable account to receive approval and exception notifications.
- A modern browser such as **Microsoft Edge** or **Google Chrome**.

 **Setup check:** If Power Automate prompts you about premium features at any point, select the option to **start a free trial** and continue with the lab.

Duration

Audience	Track A (required)	Track B (optional)	Full lab
Experienced Power Platform maker	2–2.5 hours	1.5–2 hours	3.5–4.5 hours
Fresher — new to Power Automate and Dataverse	2.5–3.5 hours	3–4 hours	6–8 hours

 **Tip:** A fresher should plan Track A as one sitting and spread Track B across a second session. The biggest time costs are Power Fx / FetchXML expressions (get them exactly right — one wrong quote fails silently) and the wall-clock wait on approval and review emails in A7, B3, and B5.

Capability coverage matrix

Every work queue capability this lab teaches, and the track that delivers it. Track A alone proves the core lifecycle; Track B completes the set.

Work queue capability	Track A (core)	Track B (self-exploration)
Create and configure a work queue	A1	—
Prioritized enqueue (priority tiers)	A3	—
Idempotency via unique reference	A3	—
Item expiry / deadline	A3	—
Cloud dequeue with FetchXML order	A5	—
Empty-queue handling	A5	—
Payload parsing and execution	A6, A7	—
	A7	—

Work queue capability	Track A (core)	Track B (self-exploration)
End-to-end processing → invoice → Processed		
Centralized monitoring	A8	B8 (deep)
Solution / ALM	A8 (basic)	B8 (env variables)
JSON input schema validation	—	B1
Processing timeout	—	B2
Business exceptions and rejection	—	B3
IT / Generic exceptions	—	B4
Requeue with delay + retry/ requeue budgets	—	B4
On hold + human remediation	—	B5
SLA at-risk / violated monitoring	—	B6
Concurrency and throughput control	—	B7
Processing history + Automation center	—	B8
Full 15-scenario test matrix	—	B8

Names used in this lab

Rename each flow and every action as you create them, using the names below. Later steps and dynamic content pickers refer to these names, and expressions reference actions by name — for example `outputs('Dequeue_Order_Item')` — so a renamed action must match the table.

Component	Name	Track
Work queue	Northwind Order Processing	A (A1)
Work queue key	NWIND-ORDERS	A (A1)
Producer cloud flow	Queue Orders	A (A2)
Processor cloud flow	Process Order Queue	A (A5)
Dequeue bound action	Dequeue Order Item	A (A5)
Parse JSON action	Parse Order Item	A (A6)
Invoice action	Create Invoice	A (A7)
Processed update action	Mark Item Processed	A (A7)
Timeout update action	Mark Processing Timeout	B (B2)
Business exception action	Mark Business Exception	B (B3)
Processing scope	Process Order	B (B4)

Component	Name	Track
Requeue update action	Requeue Order Item	B (B4)
IT exception action	Mark IT Exception	B (B4)
Remediation cloud flow	Remediate Order Exceptions	B (B5)
SLA monitor cloud flow	Monitor Order Queue SLA	B (B6)
Solution	Order Automation	A (A8)

Pre-flight setup checklist

Run this quick check before A1, so a missing prerequisite doesn't surface mid-lab:

- Power Automate (<https://make.powerautomate.com>) is open in your development environment (confirm the name in the upper-right corner). [Covered in the lab steps]
- Your account has the Environment Maker role, or another role that grants access to the work queue tables. [Refer to Appendix K]
- A Power Automate Premium license or trial is active — work queues are a premium capability. [Refer to Appendix L]
- The Northwind Traders solution is imported and seeded, so the Orders, Order Details, Order Products, and Invoices tables hold sample data.
- The Microsoft Dataverse and Approvals connections exist for the Northwind solutions (you are prompted to create them on first use).
- You can open Monitor → Work queues — this confirms work-queue access in this environment.

Understanding work queues

This section explains what a work queue is, when to use one, and how it differs from parallelism and concurrency — the concepts behind the queue you build in Track A.

What a work queue is

A **work queue** is a managed, shared list of **work items** — each one a discrete unit of work carrying its own data, priority, deadline, and status. It sits **between** the system that *produces* work and the system that *processes* it, so the two never have to run at the same time, at the same speed, or even on the same technology. Think of it as the buffer between stages on a manufacturing line: parts pile up in a tray, and the next station takes them when it is ready, in the order that matters most.

In Power Platform, a work queue and its items are **Dataverse records**. That is what gives them their four defining properties: work is **captured once** and never lost, each item is **prioritized** so the most important is done first, a failed item can be **retried or escalated on its own** without touching the rest, and the whole queue is **monitored centrally** so a fusion team can see — and remediate — everything in one place. A producer *enqueues* items; one or more processors *dequeue* and complete them; the queue orchestrator guarantees each item is handed to exactly one processor at a time.

When to leverage a work queue — and when not to

A work queue earns its keep when work must be processed **reliably, in priority order, over time** — and adds needless overhead when the job is simple, synchronous, or extremely high-throughput.

Reach for a work queue when...	Skip the work queue when...
Volume is high or bursty and must be processed dependably over time.	The automation is simple, low-volume, or a one-off — a plain flow is enough.
Items need prioritizing — high-value or urgent work first.	Every item is equal and first-in-first-out is fine with no tracking.
Each item must be retried, escalated, or audited independently.	You need a real-time, synchronous request/response answer.
Producers and processors run at different rates, times, or on different tech.	Producer and processor are the same step and never need decoupling.
You need SLA tracking, exception handling, and central monitoring.	There is no need to track item state or remediate failures.
Work is shared across multiple processors or machines.	Throughput is sub-second at massive scale — use Azure Service Bus instead.

Work queues vs. parallelism vs. concurrency

These three are often confused because all three touch “doing more than one thing” — but they answer different questions. **Parallelism** and **concurrency** are about **speed inside a single flow run**: parallel branches run unrelated actions at the same time, and concurrency (the *Degree of Parallelism* on an **Apply to each**) processes many items of one collection at once. Both are transient — they live and die within the run. A **work queue** is about **coordination across runs and systems**: it persists work in Dataverse, prioritizes it, survives failures, and decouples whoever creates the work from whoever processes it.

They are complementary, not alternatives. In fact module **B7** combines them — a scheduled work queue processor that uses concurrency to dequeue and process several items per run.

Dimension	Work queue	Parallelism	Concurrency
Primary goal	Reliability, priority, decoupling	Cut latency of unrelated steps	Speed up processing a list
Scope	Across flows, runs, and systems	Within one flow run	Within one Apply to each loop
Lifespan	Persistent (Dataverse)	Transient (one run)	Transient (one loop)
Unit of work	A business work item	Independent branches	Items of a collection
Prioritization	Yes (priority value)	No	No
Retry per item	Yes, independently	No (whole run)	No (whole run)
State and audit	Tracked on each item	None	Limited
Cross-machine	Yes	No	No
Machine scalability	Horizontal — share the queue across a machine group (or several); add machines/robots to raise throughput	None — one cloud flow run, no machine dimension	None — one loop in one run, no machine dimension

Use the decision guide below to pick the right one for a given problem.

Choosing between a work queue, parallelism, and concurrency

Figure: Choosing between a work queue, parallelism, and concurrency.

Work queue patterns in Power Automate

Because a producer and a processor are just flows, Power Automate supports five practical combinations of **cloud flows** and **Power Automate for desktop** (RPA). Which you choose depends on whether each side needs to touch a system that has **no API** — the only reason to bring a desktop flow into the picture. The queue itself is identical in every pattern; only the flows around it change.

- **1 — Cloud only:** producer and processor are both cloud flows. Everything is done with connectors and Dataverse. **This is the pattern the lab builds.**
- **2 — Desktop only:** producer and processor are both desktop flows — for legacy apps with no API on both the capture and the processing side.
- **3 — Cloud + desktop (producer side):** both producer and processor are cloud flows, and the **producer** calls a desktop flow as an action to capture work from an API-less system (for example, reading a legacy screen).
- **4 — Cloud + desktop (processor side):** both producer and processor are cloud flows, and the **processor** calls a desktop flow as an action to act on an API-less system while processing.
- **5 — Cloud + desktop (both sides):** both producer and processor are cloud flows, and **each** calls a desktop flow as an action — RPA on capture and on processing, with the queue decoupling the two robots.

The five producer/processor combinations — the queue governs the hand-off in every one

Figure: The five producer/processor combinations — the queue governs the hand-off in every one.

 **Tip:** Every pattern uses the **same** work queue and the **same** Dataverse actions to enqueue, dequeue, and update items. Adding a desktop flow only changes *how a producer captures work* or *how a processor acts on a system* — never the queue mechanics you learn in this lab. Appendix E maps each cloud action to its desktop-flow equivalent.

Pros and cons of the five patterns

The five patterns differ only in *where* Power Automate for desktop appears. The decision comes down to a single question — **does the producer's source, or the processor's target, have an API a cloud connector can call?** A desktop flow is the answer only when there is no API; it also brings a cost, because RPA needs a machine, a license, and screen automation that breaks when a UI changes. The table weighs each pattern; the guidance beneath it summarizes when to pick which.

Pattern	Pros	Cons	Machine scalability	Best for
1 — Cloud only (producer + processor cloud)	Simplest to build and maintain; runs unattended 24/7; no machine, VM, or gateway; cleanest ALM and	Limited to systems reachable by an API or connector; cannot touch legacy desktop or	Elastic — no machines or machine groups at all; scales with platform capacity, so throughput	Modern, API-first systems on both ends. The pattern this lab builds.

Pattern	Pros	Cons	Machine scalability	Best for
	lowest cost; fewest moving parts, so most reliable.	web UIs with no API.	grows without provisioning anything.	
2 — Desktop only (producer + processor desktop)	Can automate almost anything, including legacy apps with no API, on both capture and processing.	Two machines and two RPA licences; two screen-automation surfaces, so most fragile; hardest to monitor and deploy.	Bound by machine groups on both ends; scale by adding machines to both the producer and processor machine groups — the queue distributes items across them.	Fully legacy environments where neither side exposes an API.
3 — Cloud + desktop, desktop in the producer	Processing stays cloud-native — reliable, scalable, unattended — while RPA handles only the API-less capture.	Still needs a machine for the producer; capture is subject to RPA fragility and licensing.	Capture is bound by the producer's machine group; processing scales elastically in the cloud, so only the intake side needs machines.	A legacy source with a modern target (read a legacy screen, then process in the cloud).
4 — Cloud + desktop, desktop in the processor	Capture stays cloud-native — event-driven, cheap, easy — while RPA acts only on the API-less target.	Needs a machine for the processor; RPA fragility on the processing side; hybrid complexity.	Processing scales by adding machines to the processor's machine group; capture is elastic — size the group to queue depth to raise throughput.	A modern intake with a legacy target (cloud captures orders, RPA keys them into an API-less ERP).
5 — Cloud + desktop, desktop in both	Handles legacy on both ends while still gaining the queue's decoupling, prioritization, retry, and monitoring.	Highest cost and complexity; two machines and two RPA surfaces; two fragile UI-automation points; hardest to operate.	Bound by machine groups on both ends; two groups to size and scale independently — most provisioning effort of any pattern.	End-to-end legacy with no APIs, where you still want a governed, resilient hand-off.

Choosing in one line: start at pattern 1 and add a desktop flow *only* on the side that has no API — the producer (pattern 3), the processor (pattern 4), or both (pattern 5). Reach for pattern 2 only when the whole process is legacy. Every desktop flow you add buys reach at the price of a machine, licensing, fragility, and harder operations — so add it deliberately, not by default.

 **Important:** Whichever pattern you choose, the work queue's value is the same: it decouples the two sides, prioritizes work, retries failures, and gives one monitored view. The patterns change only *how work is captured and acted on* — never *how the queue governs it*.

Track A — mandatory: the core work queue

This track is required, and it stands on its own. By the end of **A8** you have a complete cloud automation: a producer that captures prioritized orders, a processor that dequeues and executes them, and a monitored queue that shows each order move from **Queued** to **Processing** to **Processed** — no desktop flow, no machine, anywhere. Every module in **Track B** extends the flows you build here, so finish this track first.

✓ **Checkpoint: Finish line:** at the end of **A8** you will have created a work queue, enqueued orders in priority order, dequeued and processed them entirely in the cloud, and watched them complete as **Processed** on the work queue page.

A1: Create and configure the work queue

A work queue is a governed container — its rules apply to every item

Figure: A work queue is a governed container — its rules apply to every item.

A work queue is a Dataverse record that holds work queue items and the rules that govern them: how long an item lives, when its SLA is at risk, what shape its data must take, and how many times it may be retried or requeued. You create it once, in the portal, before any flow refers to it. You set every option now — the core track uses the essentials, and the Track B modules exercise the rest — so you never have to come back and reconfigure.

1. Go to <https://make.powerautomate.com>.

🔧 **Setup check:** Confirm the environment name in the upper-right corner is the development environment where you imported Northwind Traders. Repeat this check every time you open a maker portal in this lab.

2. In the left navigation, select **More**, then select **Discover All**, then select **Work queues**.

In the left navigation, select More, then select Discover All, then select Work queues

3. Select **+ New work queue**.

Select + New work queue

4. In the **New work queue** side panel, enter a **Work queue name**:

Northwind Order Processing

5. In **Description**, enter:

Orders captured from Northwind Traders intake, processed in priority order by the Process Order Queue cloud flow.

6. In **Work queue key**, enter:

NWIND-ORDERS

The work queue key is a stable, human-readable identifier for the queue. Providing your own means later flows, exports, and downstream systems can refer to the queue by a name you chose rather

than a GUID. Leave it empty and the system generates one for you.

The work queue key is a stable, human-readable identifier for the queue

7. Turn on the **SLA strategy** and configure it. **Used by Track B module B6.**

- **Default time-to-live (TTL):** 4 hours
- **SLA violated after:** 4 hours
- **SLA considered at risk after:** 75%

These three settings are what turn a queue into a commitment. Any item added without an explicit expiry now expires four hours after it was enqueued, and it flips to **At risk** at 75% of that window — three hours in.

8. Set the schema type to **JSON**, select **Add schema**, and paste the schema below. **Used by Track B module B1.**

```
{
  "type": "object",
  "properties": {
    "OrderId": { "type": "string", "required": true },
    "OrderNumber": { "type": "string", "required": true },
    "OrderValue": { "type": "number", "required": true },
    "PaymentType": { "type": "string" },
    "ShipCity": { "type": "string" },
    "ShipCountry": { "type": "string" },
    "Source": { "type": "string" }
  }
}
```

This is the queue's contract. From now on, the platform validates every item's **Input** against this schema **at enqueue time** — a producer that forgets `OrderValue`, or sends it as text, is rejected at the door rather than three steps into the processor.

This is the queue's contract

 **Important:** Once a schema is added to a work queue it **can't be changed** — this protects against data inconsistencies and processing failures — so paste it exactly as shown; if it's wrong, you must create a new queue. Mark each mandatory field with `"required": true` **inside that field**:

```
"OrderValue": { "type": "number", "required": true }
```

Don't list required fields in a separate `"required": [ ... ]` array — it won't be enforced.

Don't list required fields in a separate array — it won't be enforced

Finally it would look like below,

Finally it would look like below

9. Leave **Auto-retry on IT exception off**.

 **Important:** **Auto-retry on IT exception** drives the auto-retry pattern for *desktop-flow* processing, where a machine holds the item and retries it in place. This lab has no machine, so the setting does nothing here — module **B4** implements the cloud equivalent explicitly, with a requeue and a delay

you control.

10. Turn **Allow update item input while in processing on**. Used by Track B module B5.

Turn Allow update item input while in processing

11. Select **Create**.

12. Open the new queue from the list, select **Advanced details** on the work queue details card, and use the copy icon to copy the **Work queue ID**. Keep it on your clipboard or in a scratch file — every flow in this lab needs it.

Open the new queue from the list, select Advanced details on the work queue details card

Open the new queue from the list, select Advanced details on the work queue details card



Tip: The work queue ID is also visible in the browser address bar on the work queue details page. Wherever this lab shows `<WORK-QUEUE-ID>`, paste that GUID.



Checkpoint: Northwind Order Processing appears in **Monitor > Work queues** with an SLA strategy, a JSON input schema, and you have copied its work queue ID.

Step 13:

1. For the complete list of other writable columns/attributes for a work queue check this [link](#).
2. To understand the default values. Select the Tables menu item in the left navigation pane

B) To understand the default values

B) To understand the default values

Which would open the powerapps portal, in which select the all option to see all tables.

Which would open the powerapps portal, in which select the all option to see all tables

Which would open the powerapps portal, in which select the all option to see all tables

Scroll to the bottom and find the **workqueue** table. Select the table to open in a new tab.

Scroll to the bottom and find the workqueue table

Scroll to the bottom and find the workqueue table

Below view would be visible,

Below view would be visible

Below view would be visible

In which click on the 35 more to get the popup window. In the popup window search for item, select the below items to understand the default values.

- Item maximum retry count
- Item maximum requeue count

Item maximum requeue count

Item maximum requeue count

A2: Build the producer flow

The producer captures the order and measures its urgency — nothing more

Figure: The producer captures the order and measures its urgency — nothing more.

The producer's only job is to capture work. It does not approve, invoice, or decide anything about the order beyond how urgent it is — it turns an order into a queue item and stops. That separation is the whole point: intake stays fast and cheap no matter how slow processing becomes. You build it on the same trigger Module 1 used, so an order entered in the Northwind Orders app flows into the queue exactly as it previously flowed into the approval.

1. Navigate to the Northwind Solutions Select + New > **Automation** > **Cloud Flow** > **Automated cloud flow**.

Navigate to the Northwind Solutions Select + New > Automation > Cloud Flow > Automated cloud

2. In **Flow name**, enter `Queue Orders`.
3. In the trigger search box, enter `When a row is added, modified or deleted`, select the **Microsoft Dataverse** trigger with that name, and select **Create**.

In the trigger search box, enter , select the Microsoft Dataverse trigger with that name

4. Configure the trigger parameters:

- **Change type:** **Modified**
- **Table name:** **Orders**
- **Scope:** **User**

5. Leave **Select columns** empty, and in **Filter rows**, enter:

```
nwind_orderstatusid eq 0
```

`0` is the numeric value of the **New** choice, so the producer captures an order the moment it is submitted — the same event Module 1's approval flow listened for.

Is the numeric value of the New choice, so the producer captures an order the moment

 **Important:** If your flow never triggers, check who owns the Order row. With **User** scope, orders created by anyone else are silently ignored — switch to **Organization** if other people will create orders.

6. Add a **List rows** action, rename (select the 3 dot context menu to find rename option) it to `List Order Details`, set **Table name** to **Order Details**, and in **Filter rows**, enter:

```
nwind_OrderID/nwind_ordersid eq @{triggerOutputs()?['body/nwind_ordersid']}
```

Add a List rows action, rename (select the 3 dot context menu to find rename option)

7. Add an **Initialize variable** action, rename it to `Init Order Value`, and configure it:

- **Name:** `Order Value`

- **Type:** Float
- **Value:** 0

Screenshot for A2: Build the producer flow

8. Add an **Apply to each** loop, rename it to **Calculate Order Value**, and set its input to **List of items** from **List Order Details**.

Add an **Apply to each** loop, rename it to , and set its input to List

9. Inside the loop, add an **Increment variable** action, rename it to **Increment Order Value**, set **Name** to **Order Value**, and in **Value** enter the expression:

```
mul(item()?['nwind_quantity'], item()?['nwind_unitprice'])
```

Inside the loop, add an **Increment variable** action, rename it to , set **Name** to Order

Inside the loop, add an Increment variable action, rename it to , set Name to Order

1. Click on **Save** to save the progress

✓ **Checkpoint:** The producer reads the triggering order's line items and tallies the order's total value into **Order Value** — the same calculation as Module 1, now used to decide *urgency* rather than to decide an approval.

The producer reads the triggering order's line items and tallies the order's total value into Order

The producer reads the triggering order's line items and tallies the order's total value into Order

A3: Enqueue the order as a work queue item

An item carries what the work is, how urgent it is, when it is due

Figure: An item carries what the work is, how urgent it is, when it is due, and whether it is ready.

A work queue item carries four things that matter: **what** the work is (its input), **how urgent** it is (its priority), **when it must be done by** (its expiry), and **whether it is ready** (its status). This module sets all four from the order that triggered the flow.

1. Below the **Calculate Order Value** loop, add a **Microsoft Dataverse Add a new row** action and rename it to **Enqueue Order Item**.
2. In **Table name**, select **Work Queue Items**.

In **Table name**, select Work Queue Items

3. Open **Advanced parameters** and select these fields: **Work Queue**, **Input**, **Priority**, **Unique Id or reference**, **Expiry Date**, **Processing Notes**, **Status**, **Status Reason**.

Open **Advanced parameters** and select these fields: Work Queue, Input, Priority, Unique Id or reference, Expiry

4. In **Name**, enter:

```
NW-@{triggerOutputs()?['body/nwind_ordernumber']}
```

Naming the item after the order number is what makes the work queue page readable — without a name, the item list shows the internal work queue item ID instead.

Naming the item after the order number is what makes the work queue page readable —

5. In **Work Queue Id**, enter the lookup in `/SetNameOfTheTable(identifier)` format — the same syntax Module 1 used to link an invoice to its order:

```
/workqueues (<WORK-QUEUE-ID>)
```

In Work Queue Id, enter the lookup in format — the same syntax Module 1 used



Note: when referencing other tables, that we need to use the “set name” of that table, and not the logical name. The set name of the orders table for example is “nwind_orderses”.

6. In **Input**, build the item's payload:

```
{
  "OrderId": "@{triggerOutputs()?['body/nwind_ordersid']}",
  "OrderNumber": "@{triggerOutputs()?['body/nwind_ordernumber']}",
  "OrderValue": @{variables('Order Value')},
  "PaymentType": "@{triggerOutputs()?['body/nwind_paymenttype']}",
  "ShipCity": "@{triggerOutputs()?['body/nwind_shipcity']}",
  "ShipCountry": "@{triggerOutputs()?['body/nwind_shipcountryregion']}",
  "Source": "Northwind Orders app"
}
```

In Input, build the item's payload



Important: `OrderValue` is deliberately **not** wrapped in quotation marks — the schema you added in A1 declares it as a number, and a quoted value is a string. Quote it by accident and the enqueue fails schema validation. That is the contract doing its job.

7. In **Priority**, open the expression editor (fx) and enter:

```
if(greater(variables('Order Value'), 10000), 1, if(greater(variables('Order Value'), 1000), 2, 3))
```

In Priority, open the expression editor (fx) and enter

Priority determines the pick order: **a lower value is a higher priority, with 1 being the highest**. The expression maps Module 1's approval thresholds onto three tiers — orders above 10,000 USD are picked first, orders above 1,000 USD next, everything else last. At month end, that single expression is the difference between invoicing the 40,000 USD order first and invoicing it four hundred orders later.

8. In **Unique Id or reference**, enter:

```
@{triggerOutputs()?['body/nwind_ordernumber']}
```

This value must be unique within the queue, which turns it into an idempotency key. Module 1's trigger fires again every time you save an order that is still **New** — with a unique reference, the second save's enqueue is rejected by the platform instead of quietly creating a duplicate order item that would be approved and invoiced twice.

This value must be unique within the queue, which turns it into an idempotency key

 **Tip:** Leave **Unique Id or reference** empty and the system generates a value in the format `system-<GUID>`. Unique, but meaningless — and no protection against duplicates.

9. In **Expiry Date**, open the expression editor and enter:

```
addHours(utcNow(), 4)
```

In **Expiry Date**, open the expression editor and enter

This sets the deadline explicitly on the item. Leave it empty and the item inherits the queue's **SLA violated after** value from A1 — four hours, the same answer. Setting it here shows the override, and Track B modules **B2** and **B6** rely on it.

10. Set **Status** to **Queued** and **Status Reason** to **Queued**.

Queued is the only state from which an item can be dequeued. The alternative at creation time is **On hold** — used when an item needs review or preprocessing before it is allowed to be picked up. Track B module **B5** puts items into that state deliberately and brings them back.

Queued is the only state from which an item can be dequeued

11. Select **Save**, then **Publish**.

 **Checkpoint:** **Queue Orders** ends with **Enqueue Order Item**, which writes a work queue item carrying the order's payload, a value-derived priority, an order-number unique reference, a four-hour expiry, and the **Queued** status.

Queue Orders ends with Enqueue Order Item, which writes a work queue item carrying the order's

Queue Orders ends with Enqueue Order Item, which writes a work queue item carrying the order's

Queue Orders ends with Enqueue Order Item, which writes a work queue item carrying the order's

Queue Orders ends with Enqueue Order Item, which writes a work queue item carrying the order's

A4: Test the producer

Three orders become three queued items, ordered by priority

Figure: Three orders become three queued items, ordered by priority.

Before building anything that consumes the queue, prove that the queue fills correctly and that priorities come out right.

1. Go to <https://make.powerapps.com>, select **Apps**, and play **Northwind Orders (Model-driven)**.
2. Select **Orders > New**, and select **Save** immediately to assign the order number and enable the **Order Details** subgrid.

Select Orders > New, and select Save immediately to assign the order number and enable

3. Add at least two line items totaling **more than 10,000 USD**.



Note: A single order contains multiple order items, so you add the items after you create the order.

4. Fill in **Order Date**, **Payment Type**, **Ship City**, **Ship Country/Region**, and **Notes**, set **Order Status** to **New**, and select **Save**.

If these values are not shown by default:

4.1. Publish the app and check whether the fields are visible.

4.2. To show them, modify the model-driven app in the solution. Select the app, then select **Edit in new tab**.

To show them, modify the model-driven app in the solution

Select the form, then select the pencil (edit) icon.

Select the form, then select the pencil (edit) icon

When the form opens, select the necessary fields, then select **Save and publish**.

When the form opens, select the necessary fields, then select Save and publish

5. Repeat for two more orders: one totaling **between 1,000 and 10,000 USD**, and one totaling **less than 1,000 USD**.

Repeat for two more orders: one totaling between 1,000 and 10,000 USD, and one totaling less

This custom view shows all the fields

This custom view shows all the fields. Three orders now exist, each in a different value range. If you navigate to the cloud flow, the producer flow runs and enqueues all three items.

6. Go to <https://make.powerautomate.com> > **Monitor** > **Work queues**, open **Northwind Order Processing**, and select **See all** in the work queue items section.

Go to <https://make.powerautomate.com> > Monitor > Work queues, open Northwind Order Processing, and select See all

Go to <https://make.powerautomate.com> > Monitor > Work queues, open Northwind Order Processing, and select See all



Checkpoint: Three items are listed, each named NW- followed by its order number, each with status **Queued**, and with priorities **1**, **2**, and **3** matching the three order values. Open one and confirm its **Input** holds the JSON payload and its **Expiry Date** is four hours ahead.



Tip: Want to see the schema contract reject bad work? That is Track B module **B1** — come back to it once the core loop runs end to end.

Observation exercise: watch an item move

Keep the work-queue items page open in a browser tab as you continue. An item advances through visible states — refresh the **See all** list after each step below to watch one order move through its lifecycle:

- Now (after A4): the item sits at **Queued**, at the front of the list if it is your highest-value order.
- After you run the processor in A5: its status flips to **Processing**, and **Processor Type** reads **Cloud Flow**.
- After A7 completes: it becomes **Processed**, with a processing result and a **Completed On** time.
- If you feed it a bad order later (B1 or B3): you'll instead see it land in an **Exception** state, with the reason recorded on the item.

 **Tip:** No refresh button? Re-open **See all**, or sort by **Created On** (newest first). Watching the status change in place is the clearest way to internalize how a queue decouples capture from processing.

A5: Build the processor flow and dequeue an item

Each run pulls one item — the most urgent — and its status flips to Processing

Figure: Each run pulls one item — the most urgent — and its status flips to Processing.

The processor is the consumer side of the queue. It runs on its own schedule, at its own rate, and knows nothing about how the work arrived — only that there is work. That is what lets you throttle, prioritize, retry, and monitor processing without touching intake at all.

1. Navigate to the Northwind Solutions Select **+ New > Automation > Cloud Flow > Scheduled cloud flow**.
2. In **Flow name**, enter `Process Order Queue`, set it to repeat every **5 Minute**, and select **Create**.
A schedule is only one option. Work queue processing can be started by any Power Automate trigger — **manual** for on-demand runs, **automated** for an event such as an item being created, **scheduled** for a fixed cadence, or **instant** from an app or a button. This lab schedules it because a cadence is what decouples processing rate from arrival rate.
A schedule is only one option
3. Add a **Microsoft Dataverse Perform a bound action** action and rename it to `Dequeue Order Item`.
4. Configure the action:

Parameter	Value	What it does
Table name	<code>Work Queues</code>	The table the bound action is defined on.
Action name	<code>Dequeue</code>	Asks the queue orchestrator for the next available item.
Row ID	<code><WORK - QUEUE - ID></code>	The queue to dequeue from.
request	The FetchXML payload below	The dequeue order and filter.

1. In **request**, enter the stringified FetchXML query, replacing both placeholders with your own work queue ID:

 **Tip:** New to FetchXML? **Appendix G** shows four ways to acquire or generate this query — from Advanced Find's **Download Fetch XML** (no install) to hand-building it from column logical names — and explains how to wrap the raw in the {"query": "..."} form the **request** parameter expects.

```
{ "query": "<fetch mapping=\"logical\" returntotalrecordcount=\"true\"
page=\"1\" count=\"1\" no-lock=\"false\">
<entity name=\"workqueueitem\">
<filter type=\"and\">
  <condition attribute=\"workqueueid\" operator=\"eq\" value=\"<WORK-QUEUE-
ID>\"/>
  <condition attribute=\"statecode\" operator=\"eq\" value=\"0\"/>
</filter>
<order attribute=\"priority\" descending=\"false\"/>
<order attribute=\"expirydate\" descending=\"false\"/>
</entity>
</fetch>" }
```

Read the query as a sentence: *from this queue, take one item that is still **Queued**, choosing the highest priority first and, among equal priorities, the one expiring soonest.* The `count="1"` attribute is what makes each run take exactly one item; `priority` ascending honours the tiers you set in A3, because a lower priority value is a higher priority; and `expirydate` ascending breaks ties in favour of whatever is closest to its deadline — a first-expiry-first-out order.

Read the query as a sentence: from this queue, take one item that is still Queued

 **Important:** Supplying a FetchXML expression **bypasses the orchestrator's default FIFO logic**. That is what you want here — but it also means the orchestrator no longer applies item expiration and other queue settings automatically for you. Because you took control of the dequeue order, **you** become responsible for honouring the deadline — which is exactly what Track B module **B2** adds. See Appendix G to acquire or generate this query.

 **Setup check:** The `workqueueid` condition is **mandatory** in the query. Without it the request is rejected.

A dequeued item's status changes to **Processing** automatically. **Queued** is the only state an item can be dequeued from, so an item that is already processing, on hold, processed, or in exception is never handed out twice.

2. Add a **Condition** action below the dequeue and rename it to `Item Dequeued`. In the left value, open the expression editor and enter:

```
empty(outputs('Dequeue_Order_Item')?['body/workqueueitemid'])
```

3. Set the operator to **is equal to** and the right value to `true`.

Set the operator to is equal to and the right value

4. In the **True** container, add a **Terminate** action, rename it to `End Flow - Queue Empty`, and set **Status** to **Succeeded**. Everything you build from here goes in the **False** container.

An empty queue is not a failure — it is the normal state of a healthy queue most of the time. Terminating with **Succeeded** keeps the run history meaningful: red runs then mean something genuinely went wrong.

An empty queue is not a failure — it is the normal state of a healthy

5. Select **Save**, then **Test > Manually > Test**, and let one run complete.
6. Copy the entire JSON content of the **body** output from **Dequeue Order Item** — you need it as a sample in the next module.

✓ **Checkpoint:** The run dequeues the highest-priority item. On the work queue page the item's status is now **Processing**, and its **Processor Type** reads **Cloud Flow** — the platform records that a cloud flow, not a machine, picked it up.

A6: Read the item

The payload becomes named, typed values the rest of the flow can use

Figure: The payload becomes named, typed values the rest of the flow can use.

The item's **Input** is the payload the producer wrote. Parsing it gives the rest of the flow named, typed values instead of raw text — and gives the designer a dynamic content group to pick from.

1. In the **False** container of **Item Dequeued**, add a **Parse JSON** action and rename it to **Parse Order Item**.
2. In **Content**, enter:

```
@{outputs('Dequeue_Order_Item')?['body/input']}
```

3. Select **Generate from sample** and paste the payload below, then select **Done**.

```
{
  "OrderId": "00000000-0000-0000-0000-000000000000",
  "OrderNumber": "NW-1001",
  "OrderValue": 12000,
  "PaymentType": "Credit Card",
  "ShipCity": "London",
  "ShipCountry": "United Kingdom",
  "Source": "Northwind Orders app"
}
```

💡 **Tip:** You can also generate the schema from the real payload you copied at the end of A5 — using an actual dequeued item as the sample guarantees the schema matches what your own producer writes.

You can also generate the schema from the real payload you copied at the end

You can also generate the schema from the real payload you copied at the end

Everything the processor does next reads from these parsed values. Track B modules **B2** (deadline) and **B3** (validation) insert their checks between this parse and the approval you build in **A7**.

✓ **Checkpoint:** The processor parses the dequeued item's payload into named, typed values ready for the rest of the flow to use.

The processor parses the dequeued item's payload into named, typed values ready for the rest

The processor parses the dequeued item's payload into named, typed values ready for the rest

A7: Approve, invoice, and mark the item processed

The execution half — a queued order becomes an approved invoice, and the item closes as

Figure: The execution half — a queued order becomes an approved invoice, and the item closes as Processed.

This is Module 1's approval logic, moved behind the queue. The rules are identical — above 1,000 USD needs a manager, above 10,000 USD also needs an executive — but the values now come from the item's payload rather than from a trigger, and the outcome is written back onto the item.

To avoid the parallel execution competition by modules M1, it is necessary to turn off the cloud flow before the start of the Module 5. Also if the Module 2 is executed the entry would come into the work item queue by the producer part.

This module is the **execution** half of the lab: it is where a queued order becomes an invoice.

1. Below **Parse Order Item** (still in the **False** container of **Item Dequeued**), add a **Condition** and rename it to **Manager Approval Required**.
2. Configure it: **OrderValue** from **Parse Order Item** is greater than **1000**.

Configure it: OrderValue from Parse Order Item is greater than

3. In its **True** container, add **Start and wait for an approval**, rename it to **Manager Approval**, and configure it:

- **Approval type:** Approve/Reject - Everyone must approve
- **Title:** New Order above 1000 USD: Manager approval required
- **Assigned to:** the email address of the account acting as Manager
False Container stays empty
False Container stays empty

4. In **Details**, build this template from the parsed item's values:

```
Order Number: @{body('Parse_Order_Item')?['OrderNumber']}  
Payment Type: @{body('Parse_Order_Item')?['PaymentType']}  
Ships to: @{body('Parse_Order_Item')?['ShipCity']},  
@{body('Parse_Order_Item')?['ShipCountry']}  
Total value: @{body('Parse_Order_Item')?['OrderValue']} USD  
Queue item: @{outputs('Dequeue_Order_Item')?['body/name']}
```

In Details, build this template from the parsed item's values

5. Below the approval, add a **Condition** named **Check Manager Outcome : Outcome from Manager Approval is equal to Approve**.

Below the approval, add a Condition named : Outcome from Manager Approval is equal

6. In its **False** container, add a **Terminate** action named **End Flow - Manager Rejected** with **Status** set to **Cancelled**. Leave the **True** container for the executive tier.

 **Tip:** For now a rejection simply stops the run. Track B module **B3** upgrades this to record a **Business Exception** on the item, so a rejected order is searchable and remediable rather than just a cancelled run.

7. Inside **Check Manager Outcome > True**, add a **Condition** named `Executive Approval Required (OrderValue is greater than 10000)`.

Inside Check Manager Outcome > True, add a Condition named (OrderValue is greater than)

8. Inside its **True** container, add a second **Start and wait for an approval** named `Executive Approval`, configured like the manager approval but with the Executive's email and the title `New Order above 10000 USD: Executive approval required`

Inside its True container, add a second Start and wait for an approval named , configured

9. Then a `Check Executive Outcome` condition (**Outcome is equal to Approve**) whose **False** container holds a **Terminate** named `End Flow - Executive Rejected` with **Status Cancelled**.

Then a condition (Outcome is equal to) whose False container holds a Terminate named

10. Back at the top level of the **False** container of **Item Dequeued** — below **Manager Approval Required**, outside every approval condition

Back at the top level of the False container of Item Dequeued — below Manager Approval

11. Add a **Microsoft Dataverse Add a new row** action and rename it to `Create Invoice`. Set **Table name** to **Invoices**, open **Advanced parameters**, and select **Amount Due**, **Due Date**, **Invoice Date**, and **Order**:

- **Amount Due:** `@{body('Parse_Order_Item')?['OrderValue']}`
- **Due Date:** `addDays(utcNow(), 10)`
- **Invoice Date:** `utcNow()`
- **Order:** `/nwind_orderses(@{body('Parse_Order_Item')?['OrderId']})`
The order's identifier travelled inside the item's payload, which is why the invoice can be linked without re-reading the order. Orders of 1,000 USD or less skip both approval branches and arrive here directly — small orders are invoiced without approval, by design.

The order's identifier travelled inside the item's payload, which is why the invoice can be linked

1. Below **Create Invoice**, add a **Microsoft Dataverse Update a row** action and rename it to `Mark Item Processed`. Configure it:

- **Table name:** **Work Queue Items**
- **Row ID:** `@{outputs('Dequeue_Order_Item')?['body/workqueueitemid']}`
- **Status:** **Processed**
- **Status Reason:** **Processed**
- **Completed On:** `utcNow()`
- **Processing Result:** `Approved and invoiced. Invoice created for @{body('Parse_Order_Item')?['OrderValue']} USD.`

Screenshot for A7: Approve, invoice, and mark the item processed

1. Select **Save**, then **Publish**.

 **Important:** Every path through the processor **must** end by writing a terminal status onto the item. An item left in **Processing** because a path forgot to update it is invisible to the queue — it is never dequeued again and never appears in an exception view.

 **Checkpoint:** The processor approves the order against the same two-tier rules as Module 1, creates the invoice, and closes the item as **Processed** with a result that says what happened. Run the flow once per queued item, or wait for the schedule, and watch each order complete.

A8: Monitor the queue and wrap into a solution

One page shows the whole day's work; the flows are packaged for deployment

Figure: One page shows the whole day's work; the flows are packaged for deployment.

Centralized monitoring is the reason a fusion team can run this process together, and a solution is what makes it deployable. This module closes the core loop: you confirm your items processed, then package the two flows for lifecycle management.

1. Go to <https://make.powerautomate.com> > **Monitor** > **Work queues** and open **Northwind Order Processing**.
2. In the work queue items section, select **See all**, then filter the **Status** column to show **Processing** and **Processed** to confirm your dequeue and update actions worked as expected.

In the work queue items section, select **See all**, then filter the **Status** column to show

3. Open a **Processed** item and read the fields the processor wrote:

Field	What it tells you
Status / Status Reason	The outcome and its classification.
Processing Result	The message your flow wrote for whoever reads the item.
Processor Type	Cloud Flow — proof this queue was processed without a machine.
Processing User	The identity whose connection did the work.
Completed On / Processing Duration	When the item finished and how long it took.

1. Go to <https://make.powerapps.com>, select **Solutions**, and open the **Order Automation** solution from Module 1 — or select **New solution** and create it with the default publisher.
2. Select **Add existing** > **Automation** > **Cloud flow**, and add **Queue Orders** and **Process Order Queue**. If you don't see a flow, check the **Outside Dataverse** tab. Confirm both are **published**.

 **Tip:** The work queue itself is a Dataverse record, not a solution component you add here. When you move this automation to another environment, create the queue there and update each flow's work queue ID — Track B module **B8** shows the environment-variable pattern that removes that manual edit.

 **Checkpoint:** You now have a working cloud work queue. Orders are captured in priority order, dequeued and processed entirely in the cloud, invoiced, and closed as **Processed** — and the whole thing is packaged in a solution. This is the mandatory finish line. Everything in **Track B** makes it more resilient, more observable, and more production-ready.

What you built — Track A recap

You now have a complete, cloud-only work queue for Northwind orders. The diagram below shows how the pieces fit together: intake is captured once, prioritized, processed on a schedule, and closed as an invoice — all monitored from one page.

Track A end-to-end — the producer, the queue, the scheduled processor, and central monitoring

Figure: Track A end-to-end — the producer, the queue, the scheduled processor, and central monitoring.

Module	What it added
A1	The work queue and its rules — priority, SLA, JSON schema, and retry/requeue ceilings.
A2-A3	The producer: captures each new order as a prioritized, idempotent work queue item.
A4	Verification that the queue fills in priority order.
A5	The scheduled processor: dequeues the highest-priority item with FetchXML.
A6	Reads the item's payload into named, typed values.
A7	Two-tier approval, invoice creation, and closing the item as Processed.
A8	Central monitoring, and packaging the flows into a solution.

 **Track A complete.** Track B builds resilience, human remediation, SLA monitoring, concurrency, and ALM depth on top of this same foundation.

Appendices

Appendix A: Work queue item statuses

Status	Purpose	Used in this lab
Queued	Waiting to be picked up for processing. The default when adding items.	A3 (enqueue), B4/B5 (requeue).
Processing	A system or human is currently processing the item.	Set automatically by Dequeue in A5.

Status	Purpose	Used in this lab
On hold	Temporarily paused and not available for processing.	B5 (parked by the reviewer).
Processed	Successfully processed and completed.	A7 (Mark Item Processed).
Generic exception	An unspecified, unexpected error occurred.	B4 (Mark Generic Exception).
IT exception	A technical or IT-related issue occurred.	B4 (requeue budget exhausted).
Business exception	A business rule-related issue occurred.	B3 (validation and rejection).
Processing timeout	Failed to complete within the allocated time.	B2 (Mark Processing Timeout).

Appendix B: Allowed status transitions

The platform enforces these paths, so a status that isn't a legal next step simply can't be selected interactively or written at runtime.

Status	Details	Allowed transitions
Queued	The default state when items enter the queue, and the only state from which dequeuing is allowed.	Processing
Processing	The item is currently being processed.	Processed, Exception
Processed	The item completed successfully.	Queued, On hold
Exception	An exception was raised — generic, IT, or business.	Queued, On hold
On hold	A business or IT user has picked the item to review and potentially remediate.	Queued

The work queue item lifecycle — states and the transitions the platform allows

Figure: The work queue item lifecycle — states and the transitions the platform allows.

Appendix C: Key work queue and item columns

Column	Table	What it holds
<code>workqueuekey</code>	Work Queue	The unique, human-readable key for the queue.
<code>slathresholdinpercentage</code>	Work Queue	

Column	Table	What it holds
		Percentage of the violation window at which items become at risk.
<code>inputschema / inputschematype</code>	Work Queue	The schema used to validate item input at enqueue time.
<code>itemmaxretrycount / itemmaxrequeuecount</code>	Work Queue	Platform defaults (retry 0 / requeue 1000). Not exposed on the create or edit form and not maker-editable; cloud-flow processing doesn't auto-enforce them — B4 applies its own requeue budget in the flow.
<code>allowupdateinputwhileprocessing</code>	Work Queue	Whether an item's input may be updated while processing.
<code>input</code>	Work Queue Item	The actual work item data used for processing.
<code>priority</code>	Work Queue Item	Pick order — a lower value is a higher priority, with 1 the highest. (The portal's High/Normal/Low labels map to 100/200/300; this lab writes 1/2/3 directly — any integer works, and the orchestrator always dequeues the lowest value first.)
<code>uniqueidbyqueue</code>	Work Queue Item	A value unique within the queue — the idempotency key.
<code>expirydate / delayuntil</code>	Work Queue Item	The deadline, and the time before which the item can't be dequeued again.
<code>slastatus</code>	Work Queue Item	0 NotSet, 1 In, 2 At risk, 3 Out.
<code>retrycount / requeuecount</code>	Work Queue Item	How many times the item has been retried and requeued.
<code>processingresult</code>	Work Queue Item	The outcome message written by the processor.
<code>processortype</code>	Work Queue Item	0 None, 1 Cloud Flow , 2 Flow Machine.
<code>executioncontext</code>	Work Queue Item	System-managed list of processing attempts and debugging information.

Appendix D: Known limitations

Limitation	Details
Dequeuing concurrency	Keep dequeue concurrency moderate — up to five parallel dequeue operations per work queue.
Dataverse limits	Work queues are built on Dataverse, so the same service protection and API limits apply.
Throughput and scaling	Not suited to high-throughput, sub-second scenarios; consider Azure Service Bus queues for those.
DLP policy coverage	Work queue actions are reachable by design through the native Dataverse connector, which DLP can't fully block. Use Dataverse role-based access control on the underlying tables to govern cloud-flow and API-based access.
Sovereign clouds	The work queue connector applicable to Power Automate for desktop is disabled by default in GCC High and DoD environments.

Appendix E: Cloud-only notes

This lab is built entirely with cloud flows. Two work queue capabilities behave differently when a machine is involved, and it is worth knowing which is which:

Capability	With a machine	In this cloud-only lab
Dequeue	The Process work queue items action loops over items on the machine.	The Dataverse Dequeue bound action, one item per call, with FetchXML controlling the order (A5).
Auto-retry on IT exception	The queue setting lets the machine hold the item and retry it in place.	Left off. Retries are the Dataverse action retry policy inside the run, plus an explicit requeue with Delay until across runs (B4).
Add and requeue items	Add work queue item , Add multiple work queue items , and Requeue item with delay actions.	Dataverse Add a new row and Update a row on the Work Queue Items table (A3, B4, B5).
Status updates	The Update work queue item action.	Dataverse Update a row , setting Status and Status Reason (A7, B2–B5).

Appendix F: Reference

- [Work queues overview](#)
- [Manage work queues](#)
- [Process, add, update and requeue work queue items](#)
- [Trigger work queues processing](#)
- [Known limitations for work queues](#)
- [Work Queue \(workqueue\) table reference](#)

- [Work Queue Item \(workqueueitem\) table reference](#)
- [Dataverse service protection API limits](#)
- [Use FetchXML to construct a query](#)

Appendix G: Generating the FetchXML for Dequeue

The **Dequeue** bound action in module A5 takes a FetchXML query in its **request** parameter. FetchXML is Dataverse's XML query language — the same format behind model-driven app views and the **List rows** action. A query has three parts: an (the table, here `workqueueitem`), a (the conditions — the queue's ID and a `Queued` status), and (the sort — priority, then expiry). You do not have to write it by hand; here are four ways to acquire it, from no-install to pro-maker.

Method 1 — Advanced Find / Download Fetch XML (nothing to install)

- Open a model-driven app (for example the **Admin Management App**) and open the **Work Queue Items** table view.
- Open **Advanced Find** (classic) or **Edit filters**, and add conditions: **Work Queue** equals your queue, and **Status** equals **Queued**.
- Set the sort order: **Priority** ascending, then **Expiry Date** ascending.
- In classic Advanced Find, select **Download Fetch XML** — it hands you the exact element, ready to copy.

Method 2 — Build it from column logical names (most understanding)

- Find the logical names of the columns on the **Work Queue Item** table the same way the lab finds other logical names: **Tables** → open the column → **Edit** → **Advanced options** → **Logical name**. You need `workqueueid`, `statuscode`, `priority`, and `expirydate`.
- Assemble the by hand from the anatomy above: one , a with one condition per rule, and an per sort column.
- This path installs nothing and builds a real understanding of how the query maps to the queue's rules.

Method 3 — FetchXML Builder in XrmToolBox (pro-maker tool)

- Install [XrmToolBox](#) (a free community desktop tool) and connect it to your environment.
- Open the **FetchXML Builder** plugin, choose the `workqueueitem` table, and add the conditions and ordering through dropdowns.
- It generates and validates the FetchXML live, and can execute it to preview the rows that would be dequeued.

Method 4 — List rows or Copilot (draft, then copy)

- Add a temporary Dataverse **List rows** action, set the table to **Work Queue Items**, and use its **Fetch XML Query** field — or ask Copilot for “**Work Queue Items where status is Queued, ordered by priority then expiry**”.
- Copy the generated element out of that action, then delete the temporary action.

Wrapping the FetchXML for the Dequeue request

Whichever method you use, it produces a raw element. The Dequeue action's **request** parameter does not take that element directly — it takes a **stringified JSON object** whose **query** property holds the FetchXML as text, with the inner double-quotes escaped:

```
{"query": "<fetch> ... </fetch>"}
```

That is the exact shape shown in module A5's **request** step. In practice: generate the with any method above, then paste it inside the **{"query": "..."} wrapper** — remembering that the **workqueueid** condition is **mandatory**, and that a custom order (priority, then expiry) is what overrides the queue's default FIFO dequeue.

Appendix H: Troubleshooting

Common issues and their fixes. Most map to a single setting you can confirm in a minute.

Symptom	Likely cause	Fix
The producer flow never triggers	The Orders trigger uses User scope, so it ignores rows owned by others; or the status filter didn't match.	Confirm you own the order, or set the trigger Scope to Organization; verify the filter <code>nwind_orderstatusid eq 0</code> .
An item is stuck in Processing	A processor path ended without writing a terminal status.	Every path must set Processed or an Exception status; use the remediation flow (B5) or Update a row to return it to Queued.
Enqueue fails with a schema error	The Input doesn't match the JSON schema — often OrderValue sent as text, or a required field missing.	Send OrderValue as a number (no quotes); include OrderId, OrderNumber, OrderValue. The schema is draft-3 (required inside each property).
Dequeue returns nothing	No items are in Queued state, or the FetchXML workqueueid condition is wrong or missing.	Confirm items show Queued; check the work queue ID; the workqueueid condition is mandatory (Appendix G).
Runs fail with HTTP 429 / ServiceUnavailable	Dataverse service-protection limits — usually too much dequeue concurrency.	Keep parallelism at five or fewer (B7); the requeue-with-delay logic (B4) recovers throttled items.
Approval fails with ActionResponseTimeout	The synchronous caller window closed before you responded to the approval.	Have Outlook/Teams open before testing; respond promptly, or rerun and answer immediately.
Invalid FetchXML on Dequeue	The wasn't wrapped or escaped correctly in the request JSON.	Wrap it as {"query": "..."} with inner quotes escaped (Appendix G); keep the workqueueid condition.
A token or logical name doesn't resolve	A pasted expression stayed as plain text, or a logical name differs in your environment.	Re-insert via the dynamic-content picker; copy exact logical names from the column (Edit → Advanced options).

Appendix I: Glossary

Key terms used throughout this lab.

Term	Meaning
Work queue	A Dataverse-backed, shared list of work items with rules for priority, expiry, retry, and monitoring.
Work queue item	One unit of work, carrying its Input payload, priority, expiry, and status.
Producer	The flow (or system) that enqueues items into the queue.
Processor	The flow (or machine) that dequeues and completes items.
Orchestrator	The platform service that hands each queued item to exactly one processor at a time.
Enqueue / Dequeue	Adding an item to the queue / taking the next available item for processing.
Priority	A number that sets pick order; a lower value is higher priority, with 1 the highest.
Expiry (deadline)	The datetime by which an item should be processed; drives SLA and timeout.
SLA status	The item's standing against its deadline: In, At risk, or Out, computed from the queue's SLA strategy.
Requeue vs retry	Requeue returns an item to Queued (often with a delay); retry is a connector's in-run re-attempt.
Delay until	A datetime before which a requeued item is not offered for dequeue — a back-off.
On hold	A deliberately paused item, not available for processing, awaiting a person.
Exception types	Classified failure states recorded on the item: business, IT, generic, and processing timeout.
Machine group	A pool of RPA machines a desktop-flow processor can scale across.
FIFO / FEFO	First-in-first-out / first-expiry-first-out dequeue orders.
Idempotency	Ensuring a repeated enqueue of the same work doesn't create a duplicate, via a unique reference.
FetchXML	Dataverse's XML query language, used here to control the dequeue order (Appendix G).

Appendix J: Security and governance

Work queues are Dataverse records, so who can enqueue, dequeue, or remediate is governed by Dataverse [role-based access control \(RBAC\)](#) — not by the flows themselves.

- Access: a user (or a desktop machine's user) can only act on a queue they have privileges on. Use Manage access to share a queue so a second person or processor can dequeue from it.

- Ownership and scope: the producer's trigger uses User scope in this lab, so it fires only for rows you own. Widen to Organization for shared intake — but understand who can then create work.
- Least privilege: grant a processor only the privileges it needs — on the work queue and item tables, and on the business tables it writes (Orders, Invoices).
- DLP: work queue actions are reachable through the native Dataverse connector, which DLP cannot fully block. Govern cloud-flow and API access with Dataverse RBAC on the workqueue and workqueueitem tables (see Appendix D).
- Data sensitivity: an item's Input and Processing Result can carry business data, and are visible to anyone with read access to the queue — treat them like any Dataverse column holding that data.

Appendix K: Confirming the Environment Maker role (work queue table access)

Work queues and their items are Dataverse tables, so you need a security role that grants access to them. The **Environment Maker** role lets you create flows and open **Monitor > Work queues**; for full read/write on the **workqueue** and **workqueueitem** tables, **System Customizer**, **System Administrator**, or a custom role with those privileges also works. Use either check below.

Quick self-check (no admin needed)

- Sign in to make.powerautomate.com and select your development environment from the top-right environment picker.
- In the left navigation, open **Monitor > Work queues**. If the page opens and **+ New work queue** is available, you already have the access this lab needs.
- If you instead see an access or **"user needs read-write access"** error, use the admin check below, or ask your administrator to grant the role.

Confirm or assign the role (Power Platform admin center)

- Go to the [Power Platform admin center](#).
- Select **Environments**, then open your development environment.
- Select **Settings > Users + permissions > Users**.
- Search for your name, open your user record, and select **Manage security roles**.
- Confirm **Environment Maker** (or **System Customizer** / **System Administrator**) is selected. Tick it and **Save** if it isn't.

 **Note:** A tenant admin role such as Global administrator or Power Platform administrator does not by itself grant Dataverse data access — a matching Dataverse security role is still required in the environment. See [Security roles for Dataverse](#) and [Configure user security in an environment](#).

Appendix L: Checking a Power Automate Premium license or starting a trial

Work queues are a premium capability, so your account needs a **Power Automate Premium** license — or an active free trial. Confirm what you have, and start the 90-day trial if you need it.

Check whether you already have Premium

- In [Power Automate](#), select the gear (**Settings**) in the top-right, then **View my licences** to see the plans assigned to you.
- Or open myaccount.microsoft.com > **Subscriptions** and look for **Power Automate Premium**.
- If Premium (or a trial) is listed, you're ready — skip the trial steps below.

Start the free 90-day trial

- In make.powerautomate.com, select **Try free** in the upper-right corner. When a premium feature such as work queues prompts you, you can also select the start-trial option there.
- Follow the prompts to activate the **Power Automate Premium** trial, then return to the lab.

 **Note:** If you see a message that self-service sign-up is disabled, or one asking you to contact IT, your administrator controls licensing — ask them to assign a Power Automate Premium license. See [Sign up and use paid features](#) and [Power Automate licensing](#).